

Universidad
Rey Juan Carlos

Escuela Técnica Superior de Ingeniería Informática

Grado en Matemáticas
Curso 2025-2026

Diseño y Análisis de Algoritmos

PRÁCTICA 2: VUELTA ATRÁS

Autores: GRUPO 1. Daniel Rodríguez Fernández, Diego Caballero Sánchez

Mayo 2026

Resumen

Este trabajo consiste en el planteamiento, la implementación y el análisis de un algoritmo del tipo vuelta-atrás.

Palabras clave:

- Vuelta-Atrás
- Algoritmos
- Robot
- Tornillo
- Laberinto
- Java
- Complejidad

Índice general

1. Introducción **1**

2. Objetivos **2**

3. Descripción y resolución del problema **3**
3.1. Algoritmo 4
3.2. Pseudocódigo — Clase `Robot` 4
3.3. Clases implementadas 8

4. Análisis Detallado de Complejidad **9**
4.1. Estudio teórico 9
4.2. Estudio empírico 12

5. Ejemplos de funcionamiento **17**
5.1. Modo de Ejecución 1: Interfaz por Consola (Interactivo) 17
5.2. Modo de Ejecución 2: Procesamiento por Lotes (Ficheros) 19
5.2.1. Ejemplo de tablero sin solución 20

6. Alternativas de algoritmos **22**
6.1. Uso de IA 24

7. Conclusiones **25**

8. Trabajos futuros **26**

1

Introducción

El presente documento detalla el diseño, la implementación y el análisis de una solución computacional para el problema de un laberinto. Esta práctica se enmarca dentro de la asignatura de **Diseño y Análisis de Algoritmos**, con el objetivo de profundizar en la aplicación de estrategias vuelta-atrás.
Para ello, se ha utilizado el lenguaje de programación Java.

2

Objetivos

1. **Desarrollo** Desarrollar un algoritmo de búsqueda en un tablero con casillas restringidas (análogo a un laberinto).

2. **Algoritmos vuelta-atrás** Aprender sobre la implementación de algoritmos vuelta-atrás en Java, y en qué casos estos son efectivos.

3. **Análisis de complejidad** Analizar la complejidad temporal del algoritmo desarrollado.

3

Descripción y resolución del problema

Enunciado del problema

Ün robot se mueve en un edificio en busca de un tornillo. Se desea diseñar un algoritmo que permita al robot encontrar el tornillo y regresar posteriormente a la salida. El edificio se representa mediante una matriz de tamaño $N \times M$, cuyas casillas pueden contener uno de los siguientes valores: - L: paso libre (el robot puede moverse) - E: paso estrecho (el robot no puede pasar) - T: casilla donde se encuentra el tornillo El robot comienza en la posición $[0, 0]$ y puede moverse en las direcciones $Norte$, Sur, $Este$ y $Oeste$ siempre que la casilla de destino no sea un paso estrecho y esté dentro de los límites. El objetivo es encontrar un camino desde la posición inicial hasta la casilla que contiene el tornillo. Una vez encontrado, el algoritmo debe devolver el camino de regreso desde el tornillo hasta la salida. Por lo tanto la salida del programa es el camino de regreso. No es necesario encontrar el camino más corto, sino cualquier camino válido."

3.1. Algoritmo

Para resolver este problema, se ha implementado un algoritmo del tipo vuelta atrás. Se ha decidido hacerlo así, ya que el vuelta-atrás es una estrategia de resolución que nos permite explorar posibles caminos dentro del tablero hasta encontrar alguno que lleve hasta el tornillo. En cualquier caso, como se explica más adelante, nos hemos desviado un poco del esquema vuelta-atrás para mejorar la eficiencia, por lo que podría considerarse más apropiado utilizar el término .ªlgoritmo de búsqueda en profundidad"
Necesariamente, el caso base será encontrar el tornillo. El método recursivo devolverá una variable booleana que codificará si ha encontrado el tornillo en la recursión o no. Asimismo, se recogerá el camino recorrido en una variable global (una pila, ya que se adapta muy bien a lo que pide el enunciado).
Tal y como establece el enunciado, los movimientos posibles desde cada casilla son Norte, Sur, Este, y Oeste. Por lo tanto, a priori, desde cada nodo, nuestro robot deberá explorar cada una de esas posibilidades. Sin embargo, debemos tener en cuenta que hay casillas (las E) a las que el robot no puede acceder por ser demasiado estrechas. Por lo tanto, nuestras condiciones de poda deben recoger si la casilla se puede transitar o no. Además, debemos evitar la posibilidad de recorrer un camino en círculos, ya que la implementación de un algoritmo de este tipo requiere de asumir que el problema tiene forma de árbol, es decir, que no haya bucles. Para ello, marcaremos en nuestra clase Mapa qué casillas han sido exploradas ya y cuáles no. Nuestra función de poda debe evitar también explorar de nuevo casillas que ya han sido visitadas, al menos, en la misma rama.

3.2. Pseudocódigo — Clase `Robot`

Función de poda: `intentarMoverA()`

En primer lugar, se ha implementado una función que intenta moverse a la casilla solicitada. Este método actúa como nuestra función de poda, ya que decide si puede moverse a la casilla solicitada (si es transitable y además no ha sido explorada previamente), y devuelve falso en caso de no poder. En caso de poder moverse a la casilla, la añade a la pila con el camino recorrido (global dentro de

esta clase), y marca la casilla como visitada dentro del mapa. Incluyéndolo todo en esta función, nos aseguramos de que el robot no pueda moverse en ningún momento a ninguna casilla a la que no pueda, y que además siempre marque las casillas en las que se mueve.
(El contador de pasos no es importante, ya que solo sirve para comprobaciones de complejidad posteriores.)

Algoritmo 1 intentarMoverA(f, c)

```
fun intentarMoverA(f, c)
    si mapa.transitable(f,c) y mapa.inexplorado(f,c) entonces
        mapa.marcar(f, c) camino.apilar([f, c]) contadorPasos ← contadorPasos +1 dev verdadero
    fsi
    si no
        dev falso
    fsi
ffun
```

Inicialización del algoritmo: `buscarTornillo()`

La función *buscarTornillo*() es la responsable de inicializar el algoritmo y devolver el resultado. Como se ve en el pseudocódigo, primero se mueve a la primera casilla, y en caso de no poder, determina que esta ha sido bloqueada (es una casilla 'estrecha', E), y por lo tanto finaliza advirtiendo del problema, y devuelve $NULL$ como resultado.
Si, por el contrario, comienza con normalidad en la casilla de inicio, entonces su siguiente paso es llamar a la parte recursiva del algoritmo, el Vuelta-Atrás. Una vez terminado este, el algoritmo comprueba si ha encontrado el tornillo (no siempre es posible) y, en caso afirmativo, devuelve la pila con el camino correspondiente. En caso negativo, devuelve $NULL$.

Algoritmo 2 buscarTornillo()

```
fun buscarTornillo()
    si no intentarMoverA(INICIO[0], INICIO[1]) entonces
        Avisar: «robot atascado en la casilla inicial» dev camino vacío
    fsi
    encontrado ← buscarTornilloBT()
    si encontrado entonces
        dev camino
    fsi
    si no
        dev nulo
    fsi
ffun
```

Vuelta atrás: `buscarTornilloBT()`

A continuación se puede observar el pseudocódigo del algoritmo Vuelta Atrás, que es el núcleo de este programa. En primer lugar, se comprueba la cima de nuestro camino, que nos indica cuál es la casilla donde se encuentra el robot actualmente.

En segundo lugar, se comprueba en el mapa si nos encontramos en el caso base: haber encontrado el tornillo. En caso afirmativo, nuestro algoritmo termina, devolviendo verdadero.

De no estar en el caso base, iteramos sobre todos los movimientos que el robot puede realizar hacia las casillas cercanas, que son cuatro, pero solo hasta que el booleano *encontrado* sea verdadero; de lo contrario, estaríamos haciendo movimientos innecesarios y sobrescribiendo el camino. Para cada movimiento, el robot intenta moverse, llamando a la función 1, y si puede, hace la llamada recursiva. Si después de esta, no ha encontrado el tornillo, deshace el último paso de la pila, para poder cambiar de rama.

En caso de que no haya tornillo, o de que este sea inaccesible desde la casilla de inicio, al ser nuestra función de tipo *boolean*, simplemente devolverá $false$. El programa recorrerá en profundidad todas las posibilidades, en un número finito de pasos, que dependen de los bucles for, y al terminar simplemente todas habrán devuelto $false$.

Algoritmo 3 buscarTornilloBT() — Backtracking recursivo

```
fun buscarTornilloBT()
    (f, c) ← cima(camino)
    // Caso base: solución encontrada
    si mapa.hayTornillo(f, c) entonces
        dev verdadero
    fsi
    // Explorar los 4 movimientos posibles mientras no se haya
       encontrado
    encontrado ← falso   MOVS ← {(+1, 0), (0, −1), (−1, 0), (0, +1)}
    para cada (Δf, Δc) ∈ MOVS y no encontrado hacer
        f′ ← f + Δf;   c′ ← c + Δc
        si intentarMoverA(f′, c′) entonces
            encontrado ← buscarTornilloBT()
            // Poda: si no encontrado, deshacer
            si no encontrado entonces
                camino.desapilar()
            fsi
        fsi
    fpara
ffun
```

Nótese que en ningún momento desmarcamos las casillas visitadas del mapa cuando cambiamos de rama. Esto es algo intencional, porque hace el algoritmo mucho más eficiente. Notemos que, para cada casilla que se ha explorado, se han explorado también todos los posibles caminos desde la misma, intentando llegar al tornillo. Si no se ha conseguido llegar, por lo tanto, eso significa que no hay ningún camino que pase por esa casilla y termine en el tornillo. En consecuencia, no merece la pena revisitar las casillas ya exploradas, incluso cuando cambiamos de rama dentro del backtracking. Es por eso que cuando deshacemos los pasos previos tras la recursión, no lo hacemos con las casillas marcadas en el mapa.
Asumiendo que el tablero es de tamaño $N \times M$, esta modificación impone que el número de pasos totales posible sea como máximo $N \cdot M$, el total de casillas en el tablero, y reduce considerablemente la complejidad temporal, como se verá más adelante. El lector puede pensar en este algoritmo como recorrer un laberinto impreso en papel. Podríamos marcar con un bolígrafo desde el inicio un camino cualquiera hasta su final (recorrido en profundidad), y en caso de no llegar a la casilla deseada, volver atrás desde el final hasta encontrar la primera bifurcación.

En ella se toma otro camino distinto y se repite el proceso, pero no se vuelve a pasar con el bolígrafo sobre las líneas ya dibujadas, ya que visualmente es evidente que de esa forma no hay ganancia posible.

3.3. Clases implementadas

Para desarrollar el algoritmo, se ha diseñado una estrucutra basada en programación orientada a objetos.
En primer lugar, una clase de `Entrada`, y otra de `Salida`, encargadas de gestionar los archivos de entrada y salida de datos. En segundo lugar, una clase `GestorErrores`, que se encarga de lidiar con cualquier posible excepción que ocurra en tiempo de ejecución. Complementariamente a esta última, una clase `ExcepcionPropia` que nos permite distinguir aquellos errores esperados dentro de la ejecución del código (por ejemplo, que el usuario introduzca un valor de formato no válido) de aquellas excepciones inesperadas.
Por otra parte, la estructura del código incluye una clase `Robot`, que lleva a cabo el algoritmo principal antes descrito. `Robot` tiene a su vez una pila, donde almacena los puntos que ha recorrido en el camino que recorre, y un objeto de la clase `Mapa`, donde se almacenan dos arrays bidimensionales: uno para registrar qué casillas ya fueron visitadas, y otro para recoger el tipo de casilla que le corresponde a cada una (con un enumerado `TipoCasilla`, que recoge los tres tipos).

4

Análisis Detallado de Complejidad

4.1. Estudio teórico

1. Fase de Preparación y Tratamiento de Datos

La ejecución del programa comienza con la gestión de la entrada a través de la clase `Entrada`. Esta fase es crítica porque establece el tamaño del espacio de búsqueda que el robot explorará posteriormente. El método `leerTablero` debe procesar cada una de las celdas de la matriz de entrada para validar que los valores sean correctos (solo se permiten -1, 0 o 1) y asignarlos a la estructura de datos interna.

Este proceso se realiza mediante un bucle anidado que recorre las N filas y M columnas del tablero. Internamente, la clase `Mapa` crea dos matrices adicionales: una de objetos `TipoCelda` para representar la lógica del terreno y otra de valores booleanos para gestionar las celdas exploradas. Dado que el número total de operaciones de instanciación y validación crece de forma proporcional al área del

tablero, definimos la complejidad de esta fase en función del número total de celdas n ($n = N \times M$). Al no existir bucles dependientes más allá de la lectura de la matriz, el coste es lineal respecto al número de elementos de entrada.

$$T_{init}(n) \in O(n)$$

2. Análisis del Algoritmo de Búsqueda (Backtracking)

La resolución del problema se basa en un algoritmo de búsqueda en profundidad con retroceso, implementado en la clase `Robot`. A diferencia de una búsqueda exhaustiva sin control, este algoritmo utiliza una técnica de "podaïmplícita mediante el registro de celdas visitadas, donde el método `intentarMoverA` actúa como un mecanismo de validación. Antes de permitir que el robot se desplace a una posición (f, c), este método verifica que la celda sea transitable y, fundamentalmente, que no haya sido explorada previamente.

Este control de estado es el pilar de la complejidad temporal del sistema. Al marcar cada casilla como explorada inmediatamente después de entrar en ella, se garantiza que el flujo de la recursión visite cada coordenada del mapa como mucho una vez. La eficiencia de esta exploración radica en que el coste asociado a cada paso recursivo es constante, denotado como $O(1)$. Esto se debe a que, al acceder a una nueva celda, el robot realiza un número limitado y fijo de operaciones: la comprobación de transitabilidad del terreno, el marcado de la posición en la matriz de explorados y la inserción de la coordenada en la pila de camino.

Incluso considerando las cuatro ramificaciones posibles —Norte, Sur, Este y Oeste— definidas en la estructura de control, este número de intentos de movimiento es invariable y no depende del tamaño total del tablero. Por tanto, el trabajo realizado en cada nodo o celda es $O(1)$. Esta estructura impide que el algoritmo entre en bucles infinitos o procese caminos redundantes. En consecuencia, en el peor de los casos —donde el robot debe recorrer todo el laberinto para encontrar el tornillo o confirmar que es inalcanzable— el tiempo de ejecución se reduce a la suma de trabajos constantes sobre el número total de celdas. Esto confirma que la complejidad de la fase de búsqueda escala linealmente con el número de celdas transitables del tablero:

$$T_{busq}(n) \in O(n)$$

Observación

Esta complejidad es consecuencia de una modificación en el esquema general de los algoritmos vuelta-atrás. Usualmente, los cambios realizados en una rama deben borrarse al cambiar a otra. Si en este caso no dejásemos marcadas las casillas visitadas entre ramas, la complejidad temporal pasa a ser exponencial, como es habitual en este tipo de algoritmos. En particular, si el algoritmo fuera un vuelta-atrás puro, la profundidad de la recursión sería de n, el número de casillas, y en cada paso, el número de casillas que el robot puede visitar en el peor de los casos es 3 -los movimientos posibles salvo el de regreso a la casilla desde la que vino- lo que generaría una complejidad de orden $O(3^n)$.

3. Síntesis de la Complejidad Temporal Final

La complejidad total del sistema $T(n)$ se obtiene mediante la suma de los costes de inicialización y búsqueda. Según las reglas de la notación asintótica, cuando sumamos dos funciones de la misma clase de complejidad, el resultado se mantiene en dicha clase:

$$T(n) = T_{init}(n) + T_{busq}(n) \in O(n) + O(n) \Rightarrow T(n) \in O(n)$$

Es fundamental realizar una distinción técnica según los parámetros de entrada. Si el análisis se realiza basándose en la dimensión del lado L de un tablero cuadrado ($n = L \times L$), observamos que el tiempo de ejecución crece de forma cuadrática respecto a la longitud del lado. Esta es la observación que se pretende documentar en los informes de rendimiento generados por la clase `Test`, donde se espera que, al duplicar el lado del tablero, el tiempo medio de ejecución se cuadruplique.

$$T(L) \in O(L^2)$$

4.2. Estudio empírico

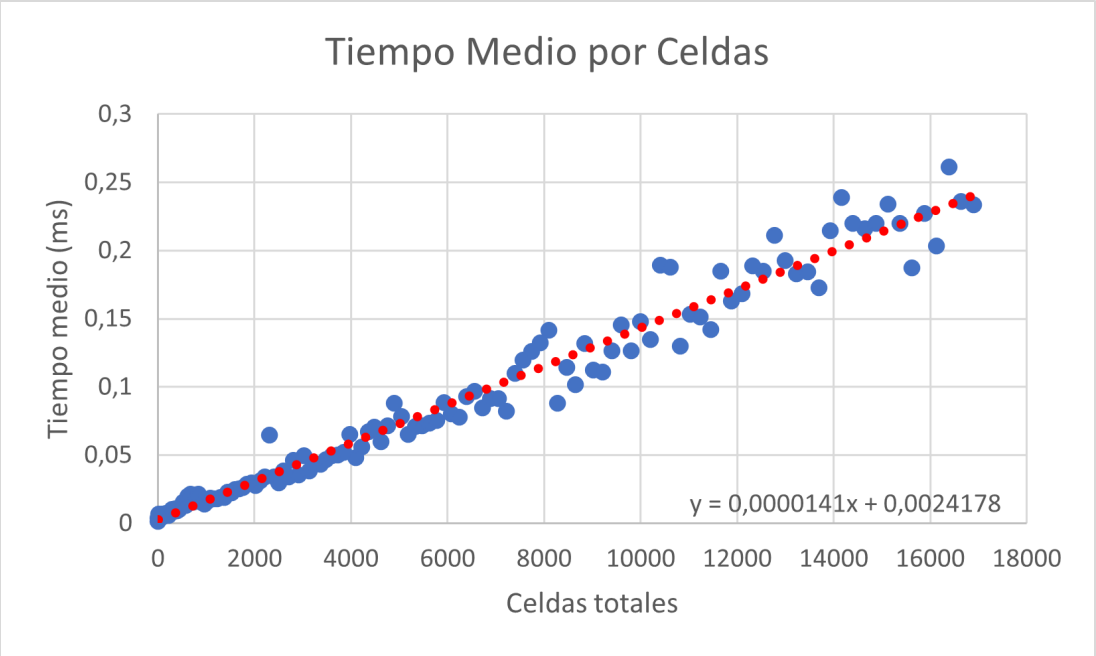
Una vez establecido el marco teórico que sitúa la complejidad del algoritmo en $O(n)$ (siendo n el numero de celdas), es imperativo validar dicho comportamiento mediante una fase de experimentación controlada. El estudio empírico permite contrastar si las hipótesis matemáticas se mantienen bajo las restricciones reales de ejecución, tales como la gestión de memoria de la JVM y los tiempos de acceso a las estructuras de datos.

Para este análisis, se ha utilizado la clase especializada `Test`, la cual automatiza la generación de escenarios aleatorios con dimensiones crecientes. El experimento se ha configurado bajo las siguientes premisas:

- **Rango de muestras:** Se han analizado tableros con un número total de celdas creciente, desde configuraciones mínimas hasta alcanzar más de 16,000 celdas, para observar la tendencia a largo plazo.

- **Mitigación de ruido:** Para cada tamaño de tablero, se han realizado múltiples iteraciones (intentos), calculando el tiempo medio de ejecución para reducir el impacto de procesos en segundo plano del sistema operativo.

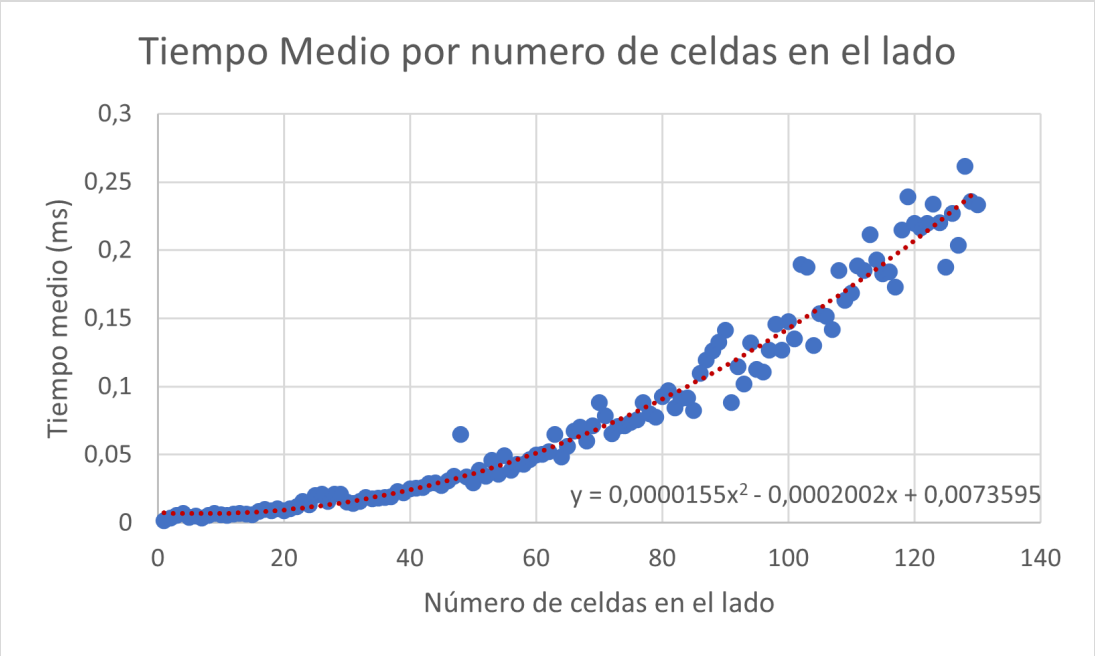
- **Instrumentación:** La medición del tiempo se ha realizado mediante la función `System.nanoTime()`, garantizando una precisión de nanosegundos para capturar el coste constante de cada paso recursivo antes de ser convertido a milisegundos para su análisis.



En esta gráfica presenta la relación entre el número total de celdas del escenario (n) y el tiempo medio de ejecución medido en milisegundos.

Al observar la distribución de los datos experimentales obtenidos por la clase `Test`, se pueden extraer las siguientes conclusiones:

- **Ajuste Lineal:** Los puntos presentan una disposición claramente lineal, lo que se confirma con la recta de regresión calculada: $y = 0,0000141x + 0,0024178$. Esta pendiente casi despreciable indica que el incremento de tiempo por cada celda adicional es constante y extremadamente bajo.
- **Validación de la Cota $O(n)$:** La ausencia de curvaturas ascendentes (típicas de complejidades cuadráticas o exponenciales) demuestra que el algoritmo de Backtracking, gracias al control de celdas visitadas, no reexplora caminos de manera redundante. Cada celda transitable se procesa una única vez, manteniendo el coste lineal respecto al área total.
- **Estabilidad del Sistema:** A pesar de las ligeras fluctuaciones en tableros de mayor tamaño ($n > 10,000$), causadas por la generación aleatoria de obstáculos que altera la profundidad de la búsqueda, la tendencia se mantiene firme sobre la línea de regresión. Esto prueba la robustez del método `intentarMoverA` en diferentes configuraciones de mapa.



En esta segunda comparativa, se analiza el comportamiento del algoritmo en función del lado del tablero (N), asumiendo escenarios cuadrados de dimensiones $N \times N$.

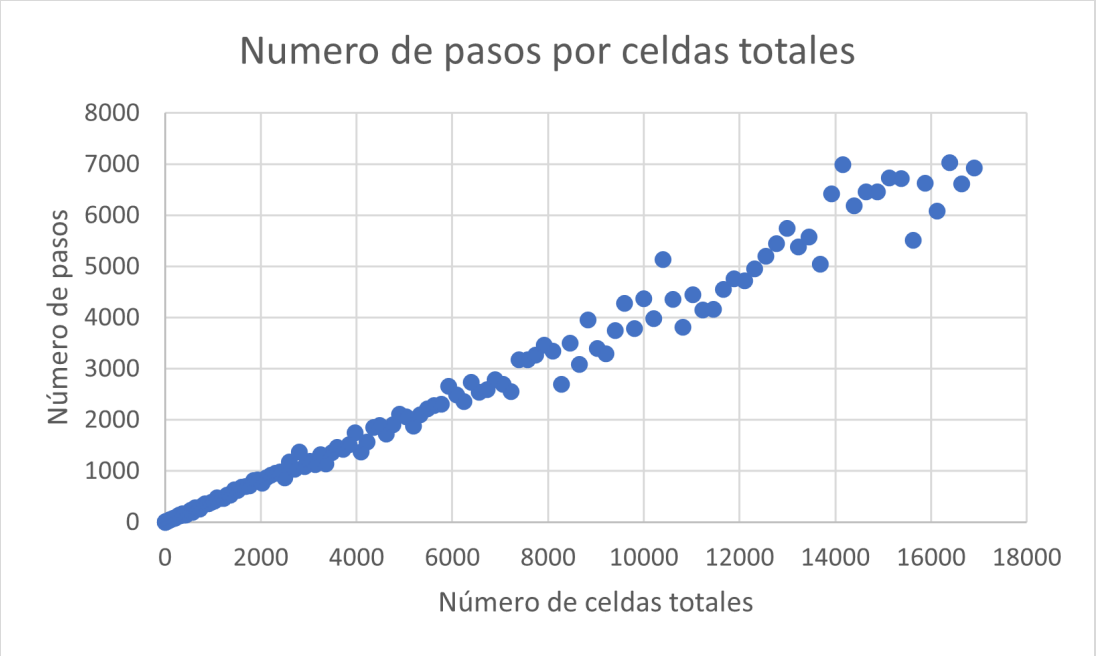
A diferencia de la gráfica anterior, aquí la tendencia no es lineal, lo que permite realizar las siguientes observaciones técnicas:

- **Crecimiento Cuadrático:** La nube de puntos describe una parábola ascendente. Esto es consistente con la teoría de algoritmos sobre matrices, donde el tamaño de la entrada es N^2. Por tanto, aunque el algoritmo es lineal respecto al número total de celdas ($O(n)$), es **cuadrático respecto al lado del tablero ($O(N^2)$)**.

- **Análisis de la Pendiente Variable:** Se observa que conforme N aumenta, el tiempo necesario para resolver el laberinto crece de forma acelerada. Esto se debe a que incrementar el lado en una unidad implica añadir $2N + 1$ nuevas celdas potenciales que el robot debe evaluar.

- **Variabilidad Aleatoria:** El incremento de la dispersión en valores altos de N (entre 80 y 100) refleja el impacto de la probabilidad de obstáculos. En tableros grandes, la disposición de las paredes puede generar caminos muy directos o laberintos altamente complejos, provocando variaciones naturales

en el tiempo medio de ejecución de la clase `Test`.



Para profundizar en el comportamiento interno del algoritmo, se ha representado el número de pasos realizados frente al número total de celdas del tablero (n). En este contexto, un paso se define como cada activación del método recursivo `intentarMoverA`, lo que equivale a la visita y procesamiento de un nodo en el espacio de estados. Del análisis de la Figura se pueden concluir las siguientes observaciones:

- **Crecimiento Lineal del Recorrido:** La gráfica muestra una tendencia lineal clara, lo que confirma que el número de nodos explorados escala de forma proporcional al área del tablero. Esto demuestra que el algoritmo no incurre en un crecimiento exponencial, validando la eficacia del Backtracking optimizado.

- **Eficacia del Marcado de Visitados:** Se observa que el número de pasos es consistentemente inferior al número total de celdas ($pasos < n$). Esto es una prueba empírica de que la matriz de `explorados` está funcionando correctamente, impidiendo que el robot evalúe la misma celda más de una vez o entre en ciclos infinitos.

- **Densidad de Búsqueda:** La dispersión observada en los valores más altos de n se debe a la naturaleza aleatoria de los obstáculos. En algunos escenarios, el robot encuentra el objetivo rápidamente, mientras que en otros debe explorar una mayor porción del mapa transitable antes de alcanzar la solución o determinar que no existe camino.
- **Correlación con la Complejidad Temporal:** La similitud morfológica entre esta gráfica y la de tiempo medio ratifica que cada paso u operación sobre un nodo tiene un coste temporal constante $O(1)$, consolidando la cota superior lineal para todo el proceso.

5

Ejemplos de funcionamiento

5.1. Modo de Ejecución 1: Interfaz por Consola (Interactivo)

El programa ha sido diseñado para detectar la ausencia de argumentos en la línea de comandos. En tal caso, se activa un modo interactivo que guía al usuario mediante mensajes por la salida estándar.(`System.out`).

Como se ilustra en la figura, el sistema solicita:

1. Elegir el modo de uso. Esta el normal que resuelve el problema pedido por el enunciado y un modo alternativo que genera un archivo con valores sobre medias de timepo de ejecución usado para el estudio d ela complejidad del algoritmo. Se tendrá que poner 1 o 2 respectivamente para elegir el modo, en el caso de la foto el 1.

2. Lo siguiente es introducir un numero entero de filas y de columnas.

3. A continuación pregunta el tipo de celda fila por fila. Hay que introducir los valores (-1,0,1) con un espacio entre medias en cada celda de las distintas columnas como se puede apreciar en la foto.

```
run:
--- Modo Teclado ---
Escoge un modo de ejecuci□n:
        1) Introducir el laberinto manualmente
        2) Ejecutar test de complejidad
Opci□n: 1
Introduzca n□mero de filas: 3
Introduzca n□mero de columnas: 3
Fila 0:
0 0 -1
Fila 1:
0 -1 0
Fila 2:
0 0 1
```

Una vez finalizada la introducción de datos, el algoritmo procesa la informacion y devuelve la solución del camino dado por puntos en fomra [,] directamente en la consola como se aprecia en la foto.

```
Camino (Tornillo ? Origen):
[2,2]
[2,1]
[2,0]
[1,0]
[0,0]
```

5.2. Modo de Ejecución 2: Procesamiento por Lotes (Ficheros)

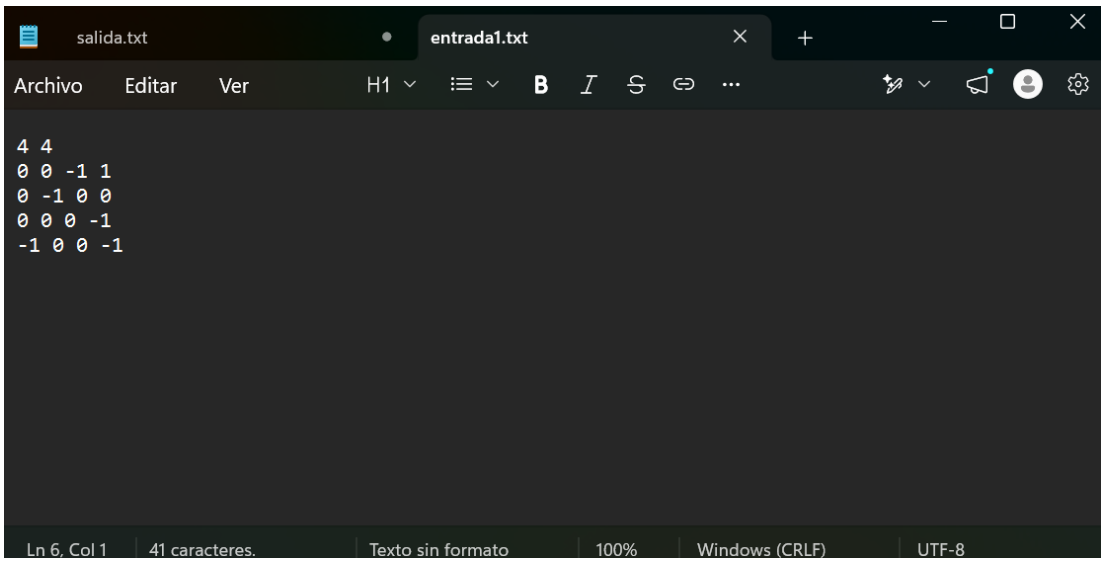
Para escenarios de alta carga de datos, el programa permite el paso de parámetros (`args[0]` y `args[1]`). Los cuales son interpretados como archivo de entrada (args[0]) y de salida (args[1]):

En la siguiente captura se muestra una sesión de la línea de comandos (CMD) donde:

- Se invoca la máquina virtual de Java pasando un fichero de entrada con un tablero 4x4. Tambien puedes introducir los datos como en el anterior ejemplo en el cmd.

  (`entrada1.txt`).

- El programa finaliza su ejecución sin errores, indicando que la lógica se ha procesado correctamente. Depende de si le pasamos un argumento (el primero es el de entrada) o dos (el segundo sería el de salida) nos da la respuesta en el cmd o en un fichero de nombre igual al segundo argumento como se parecia en las fotos.



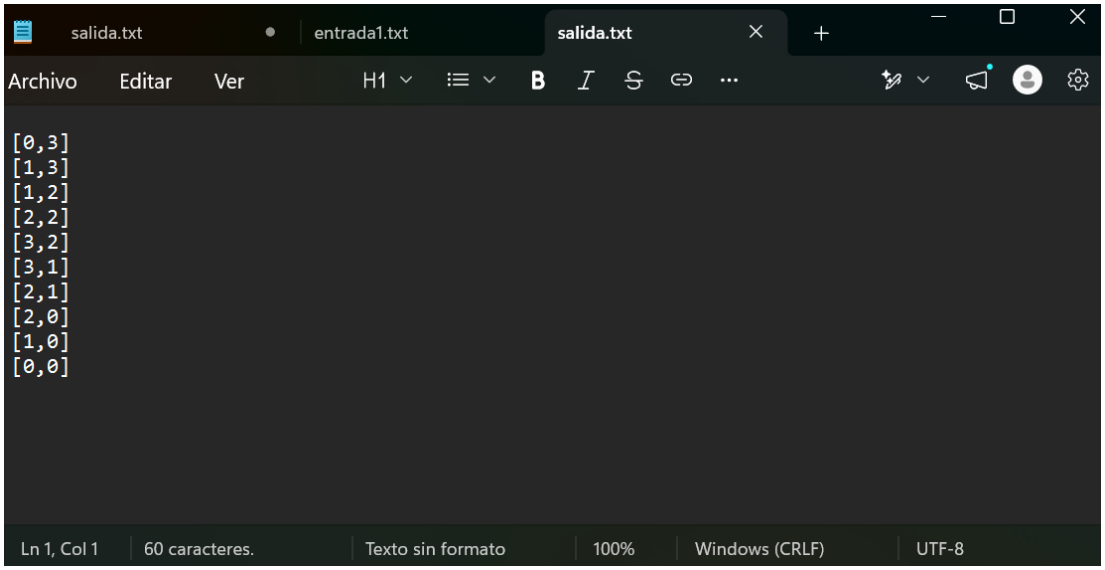


```
Camino (Tornillo ? Origen):
[0,3]
[1,3]
[1,2]
[2,2]
[3,2]
[3,1]
[2,1]
[2,0]
[1,0]
[0,0]
```

La otra opción sería meter de segundo argumento un archivo de salida, lo cual se vería:

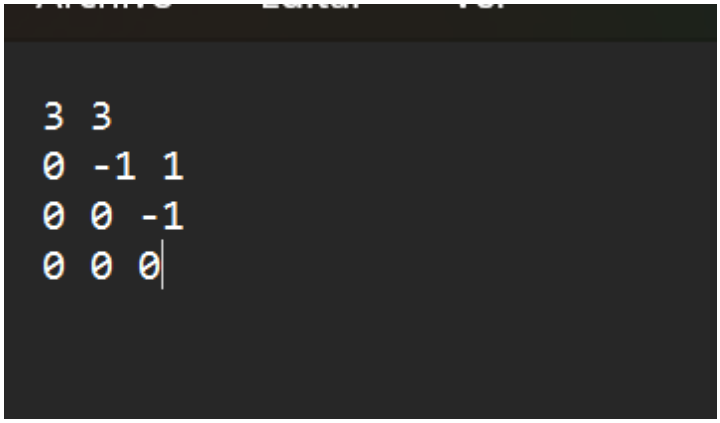
```
C:\Users\diego\OneDrive\Documentos\NetBeansProjects\RobotTornillo>"C:\Program Files\Apache NetBeans\jdk\bin\java.exe" -c
p build\classes robottornillo.RobotTornillo entrada1.txt salida.txt
Resultado guardado en salida.txt
```

De esta forma tal como pide el enunciado se deja la solución en un fichero de texto con el nombre indicado por args[1]:



5.2.1. Ejemplo de tablero sin solución

En caso de que no exista un camino viable, como en el siguiente fichero:



se mostrara un mensaje informando de que no existe dicho camino, como se puede ver aqui:

```
C:\Users\diego\OneDrive\Documentos\NetBeansProjects\RobotTornillo>"C:\Program Files\Apache NetBeans\jdk\bin\java.exe" -c
p build\classes robottornillo.RobotTornillo entrada2.txt
No se ha encontrado un camino válido al tornillo.
```

6

Alternativas de algoritmos

A continuación, resumimos algunas alternativas al algoritmo desarrollado en esta práctica:

1. Algoritmo Dijkstra. Es la única opción conocida vista en clase con la que creemos que se puede resolver el problema. Podríamos interpretar el tablero del problema como un grafo, en el que los vértices son las casillas, y dos aristas están conectadas entre ellas si el robot puede desplazarse de la una a la otra con algún movimiento válido, y su ponderación es siempre 1. Por lo tanto, es posible resolver el problema con un algoritmo de tipo voraz. La ventaja de este método es que no solo halla una solución, sino además la mejor solución posible, a diferencia del nuestro. Por otra parte, su desventaja es que resulta excesivamente complicado para un problema de este tipo.
   Como la complejidad temporal de este algoritmo es $O(|A|log|V|)$ [2], donde A es el número de aristas y V el de vértices. En el contexto de nuestro tablero, en el peor de los casos (cuando no hay prácticamente paredes), el número

de aristas es $2MN-M-N$, y el de vértices es simplemente MN, donde M es el número de filas y N el de columnas. Por lo tanto, un algoritmo tipo Dijkstra tendría complejidad $O((2MN-M-N)\log(MN)) = O(MN\log(MN))$, lo cual es algo peor que el algoritmo presentado en este trabajo.

2. Algoritmo del tipo Divide y Vencerás. No hemos conseguido plantear ningún algoritmo del tipo Divide y Vencerás que pueda resolver el problema del robot y el tornillo de forma satisfactoria. El problema principal de este método es que requiere de separar el tablero en problemas de igual naturaleza, mientras que en un problema como el nuestro, no podemos asumir que el robot deba pasar específicamente por cualquier región concreta del tablero.

3. Como alternativa complementaria a nuestro método, y con el objetivo de optimizar el camino solución, sería posible implementar un algoritmo de tipo Ramifica y Poda. En este caso, hallaríamos la primera solución con nuestro algoritmo actual, que es más rápido, y en las siguientes iteraciones utilizaríamos un esquema más parecido a un vuelta-atrás puro, que desmarque las casillas visitadas al cambiar de rama. Así, el algoritmo probaría todos los caminos posibles, pero podándolos en caso de ser más largos que la anterior solución encontrada, lo que podría agilizar algo el proceso. Por el hecho de tener que probar todos los caminos posibles más cortos que el ya encontrado, este algoritmo volvería a tener complejidad $O(3^n)$, siendo n el número de casillas.

4. Por último, buscando alternativas de algoritmos de búsqueda en laberintos, hemos encontrado `BFS`, Breadth-First Search[3], que consiste en un recorrido en anchura en lugar de profundidad. El proceso es bastante simple: los puntos no visitados visitables desde nuestra posición actual se añaden a una cola (FIFO), y las casillas ya visitadas son marcadas como tal. De esta manera, para cada "distancia"posible desde el inicio, probamos cada opción disponible, y nuestra condición de parada es también hallar el tornillo. De esta manera, la complejidad temporal sigue siendo lineal, pero nos garantizamos encontrar siempre el camino de mínimo tamaño. (En la práctica, esta resolución es prácticamente equivalente a Dijkstra).
   La complejidad de este algoritmo es lineal con el tamaño del grafo asociado al laberinto, concretamente $O(|A| + |V|)$, lo que es equivalente asintótica-

mente en nuestro caso a $O(NM) = O(n)$, igual a nuestro código, con la ventaja de que al menos encuentra siempre el camino mínimo.

6.1. Uso de IA

Se utilizó IA para resolver dudas sobre sintaxis y programación en el lenguaje Java, y se le pidió ayuda para depurar el código y encontrar bugs en el mismo, lo que nos permitió agilizar el proceso de programar, pero sobre todo el de encontrar errores escondidos en un código de relativa longitud.

7

Conclusiones

En este trabajo se ha aplicado la estrategia resolutiva de los algoritmos vuelta-atrás, modificada para mayor eficiencia temporal, al caso de un robot buscando un objeto en un laberinto. En general, se ha utilizado un algoritmo que recorre en profundidad todos los caminos desde la casilla de inicio, podando aquellos que no terminan en el tornillo y aquellas casillas que ya han sido exploradas, y parando el algoritmo cuando encuentra alguna casilla que termina en el tornillo. Además, se ha analizado su complejidad temporal, y se ha comprobado que esta es del orden $O(N \times M)$, lineal con el número de casillas totales en el tablero. Finalmente, se ha ejemplificado el algoritmo implementándolo con programación orientada a objetos en el lenguaje de programación Java, y resolviendo varios ejemplos.

8

Trabajos futuros

En trabajos futuros, podría ser interesante implementar una visualización del tablero, el camino encontrado y el algoritmo en pantalla de una manera más cómoda para el usuario.

Bibliografía

[1] Paredes Velasco, Máximo. *Apuntes del Aula Virtual*. Diseño y análisis de Algoritmos, Universidad Rey Juan Carlos.

[2] Wikipedia contributors, *Algoritmo de Dijkstra — Wikipedia, la enciclopedia libre*, `https://es.wikipedia.org/wiki/Algoritmo_de_Dijkstra` (consultado el 5 de mayo de 2026).

[3] C. Esteves, *Breadth-First Search (BFS)*, `https://www.cimat.mx/~cesteves/cursos/pa/pdf/BFS.pdf` (consultado el 5 de mayo de 2026).